Currency Converter

Compte rendu du projet Vue.js

> **Projet :** convertisseur de devises
>
> **Technologies :** Vue 3, Composition API, TypeScript et Vite
>
> **Livrable :** application responsive avec clavier numérique et fonctionnalités bonus

Objectif

Dans ce projet, j'ai réalisé une petite application qui permet de saisir un montant, de choisir deux devises et d'obtenir directement le résultat. J'ai essayé de rester proche de la capture du sujet avec une carte simple, une zone bleu pâle et un clavier numérique.

1. Choix d'interface

J'ai choisi une structure compacte pour ressembler à la maquette. La partie du haut contient les deux listes, le montant, le résultat et le bouton `CONVERTIR`. En dessous, j'ai placé le clavier dans l'ordre demandé : 7, 8, 9, puis 4, 5, 6, puis 1, 2, 3, et enfin C, 0, .

- Le bleu pâle met en évidence la zone de saisie sans réduire la lisibilité.
- Le turquoise identifie les touches numériques et le rouge distingue l'effacement.
- Les sorties de montant utilisent une grande taille de texte pour faciliter la lecture.
- La mise en page utilise des dimensions relatives pour rester utilisable sur un écran étroit.



Interface initiale.



Exemple après saisie de 120 USD.

2. Organisation du code

J'ai gardé `App.vue` très simple : il affiche le composant `CurrencyConverter.vue`. Toute la logique de l'interface est dans ce composant. J'ai mis les devises et les taux dans `src/data/taux.ts` pour pouvoir les retrouver facilement et avoir des taux de secours.

3. Logique de conversion

Pour faire le calcul, j'utilise des taux exprimés par rapport à l'euro. La formule utilisée est la suivante :

$$\text{montant converti} = \text{montant} \times \text{taux cible} \div \text{taux source}$$

Par exemple, avec les valeurs du sujet, 120 USD donnent 102,56 EUR. Grâce à la réactivité de Vue, le résultat se met aussi à jour quand je change le montant ou une devise.

Fonctionnalités bonus

- La conversion est automatique pendant la saisie et après une modification de devise.
- L'application tente de récupérer les taux depuis l'API Frankfurter au démarrage.
- Si le réseau ou l'API ne répond pas, les taux fixes du fichier de données restent actifs.
- Le clavier physique accepte les chiffres, le point, Retour arrière, Échap, C et Entrée.

4. Difficultés rencontrées et solutions

La principale difficulté a été de reproduire une interface assez petite tout en gardant des boutons faciles à utiliser. J'ai donc séparé le composant et le fichier des taux pour que le code reste compréhensible.

J'ai aussi rencontré le problème de l'API : elle peut ne pas répondre ou être bloquée par le navigateur. Pour éviter que l'application ne tombe en panne, je garde des taux fixes dans le projet. La conversion fonctionne donc même sans connexion.

Pour finir, j'ai utilisé le contrôle TypeScript et le build Vite. La commande `npm run build` vérifie que le code est correct et prépare la version de production.